

PROJECT REPORT

Recruweb Salesportal

A Three-Portal Field Sales, Commission & Payout Management Platform

Repository: github.com/Ahaan99/salesportal

Stack: Next.js 16 · Express.js · Supabase (PostgreSQL) · Google Gemini

Date: August 2026

Status: Feature-complete pilot; verified end-to-end

Table of Contents

1. Executive Summary
2. System Architecture
3. Technology Stack
4. User Roles & Portals
5. Implemented Features
6. Core Business Flow (Flowchart)
7. Data Flow Diagrams (DFD Level 0 & Level 1)
8. Database Design
9. Key Modules — How They Work
10. Security Measures
11. Testing & Verification Performed
12. Limitations & Production Roadmap
13. Appendix: Environment Variables

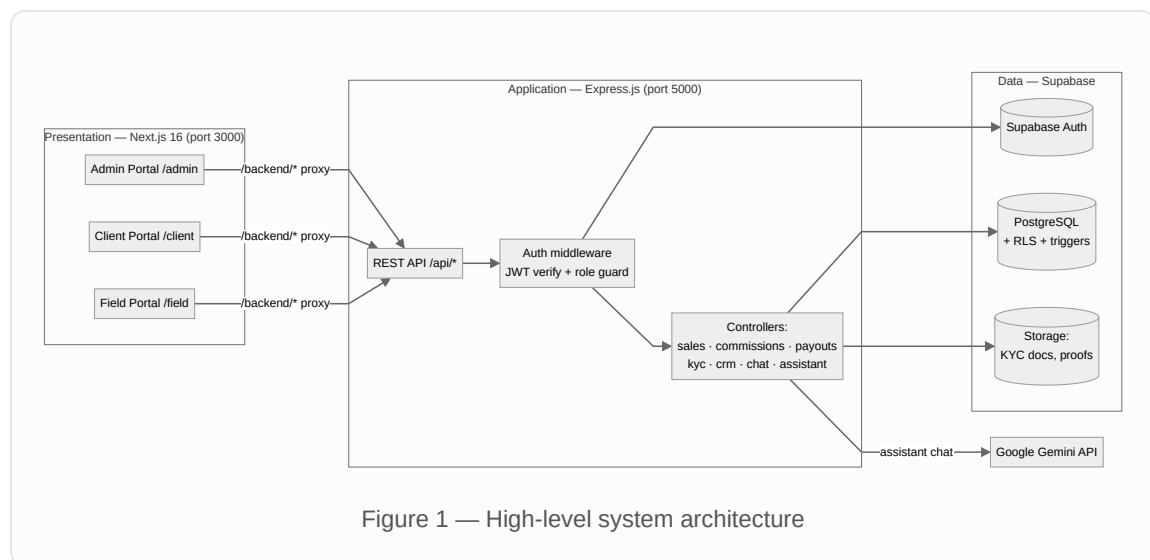
1. Executive Summary

Recruweb Salesportal is a full-stack web platform that digitizes the complete lifecycle of a field-sales operation: onboarding sales officers, recording door-to-door product sales with proof of payment, verifying those sales through an admin workflow, computing commissions automatically, and paying sellers out through a wallet-and-payout-request system.

The platform serves three distinct user groups through three role-scoped portals — **Admin** (operations command center), **Client** (product owners / brands), and **Field** (sales officers and independent sellers). Authentication and data are backed by Supabase (PostgreSQL + Auth), business logic runs on an Express.js API, and the user interface is a Next.js 16 application. An embedded AI assistant (Google Gemini) answers role-aware questions on the field and client portals.

2. System Architecture

The system follows a classic three-tier architecture with a clean separation between the presentation layer (Next.js), the application layer (Express API), and the data layer (Supabase / PostgreSQL). The frontend never talks to the database directly for business operations; all writes flow through the backend, which enforces roles and validation.



- **Proxying**: the frontend rewrites `/backend/*` to the Express server, so browsers only ever talk to one origin.
- **Session model**: Supabase Auth issues JWTs held in cookies; the Express middleware verifies the token and loads the user profile (role, seller category) on every request.

- **Database triggers:** wallet balances are maintained by PostgreSQL triggers that fire when commission or payout rows change state, keeping money math consistent even if multiple API paths touch the same data.

3. Technology Stack

Layer	Technology	Purpose
Frontend	Next.js 16 (App Router), React, Tailwind CSS, framer-motion, SWR	Three portals, responsive UI, client-side data caching
Backend	Node.js, Express.js (<code>node - -watch</code>)	REST API, business rules, role enforcement
Database	Supabase (PostgreSQL)	Relational data, RLS, triggers, functions
Auth	Supabase Auth (email + password)	Signup/login, JWT sessions, password reset
Storage	Supabase Storage	KYC documents, payment proofs (signed URLs)
AI	Google Gemini	In-app assistant for field & client users
Payments	UPI deep links + QR (<code>react - qr - code</code>)	Gateway-free payment collection from customers
Geo	Browser Geolocation API + BigDataCloud reverse geocoding	Live location detection during onboarding

4. User Roles & Portals

Role	Portal	Responsibilities
Admin	<code>/admin</code>	Command center; verify sales; manage commissions & payouts; approve KYC; manage officers, clients, products; reports; inbox; notifications. Super-admin locked to a single configured email.
Client	<code>/client</code>	Brand/product owners: manage products & inventory, view orders and performance, submit KYC, product launches, returns.
Field	<code>/field</code>	Sales officers (assigned territory) and independent sellers: record sales, collect UPI payments, track orders, leads, earnings wallet, request payouts.

Role redirection is enforced at login: each account lands only on its own portal, and API routes reject calls from the wrong role. The `profiles` table also carries a `seller_category` (`field` or `independent`) that changes commission handling and onboarding requirements.

5. Implemented Features

5.1 Field Portal

- **Guided seller onboarding** (`/field/join`) with seller-category choice and **live location detection** — real browser geolocation reverse-geocoded to a city name, with permission/timeout error handling and coordinate fallback.
- **Sell flow** (`/field/sell`): product picker with live stock, customer details with an **international phone input** (36 country codes; normalizes Devanagari, Arabic-Indic, Bengali, Tamil and other Unicode numerals to ASCII digits).
- **UPI payment collection**: when payment mode is UPI, the form renders a scannable QR code and a tap-to-pay `upi://pay` deep link carrying the payee VPA, the exact order total, currency, and an order note — compatible with GPay, PhonePe, Paytm, and BHIM. Includes copy-VPA fallback and unconfigured-VPA guard.
- **Orders, Leads, Reports** pages scoped to the signed-in seller.
- **Earnings & wallet**: commission ledger (pending → available → withdrawn), payout request dialog pre-filled with the available balance and saved bank/UPI details.
- **AI assistant widget** (bottom-left) answering commission, product, and process questions via Gemini.

5.2 Admin Portal

- **Command Center dashboard** with operational KPIs.
- **Sales Verifications**: pending queue with full sale detail (customer, product, amount, payment mode and reference), Approve / Reject with admin note.
- **Commission Management**: commissions ledger with status tabs, *Release to Available* action that credits the seller wallet; **Payout Requests** tab with *Approve & Pay* (records transaction reference) and Reject.
- **KYC review, Clients, Products, Officers** management, **Reports, Inbox** (support chat), and **Notifications**.

5.3 Client Portal

- Dashboard, **Products** and **Inventory** management, **Orders, Performance** analytics, **KYC** submission, product **Launch** workflow, **Returns**, Notifications, and Profile.

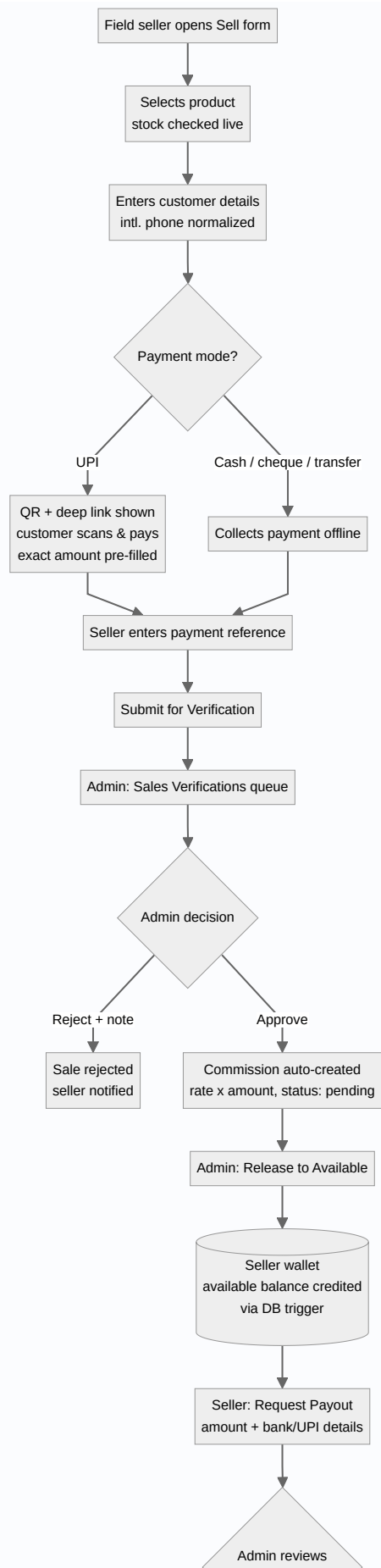
5.4 Platform-wide

- Supabase email/password authentication with role-based redirects and route guards.
- Support chat widget (bottom-right) wired to the admin inbox.

- In-app notification system.
- Robust error states on data-heavy pages (loading, empty, and failure messages).

6. Core Business Flow

The heart of the system is the money pipeline: a recorded sale must survive human verification before any commission exists; a commission must be explicitly released before it becomes withdrawable; and a payout must be approved with a bank transaction reference before it is marked paid. Every arrow below was exercised and verified in live end-to-end tests.



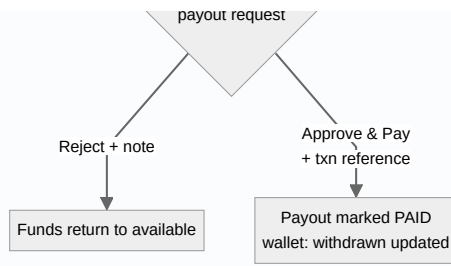


Figure 2 — Sale → Verification → Commission → Wallet → Payout flowchart

7. Data Flow Diagrams

7.1 DFD Level 0 — Context Diagram

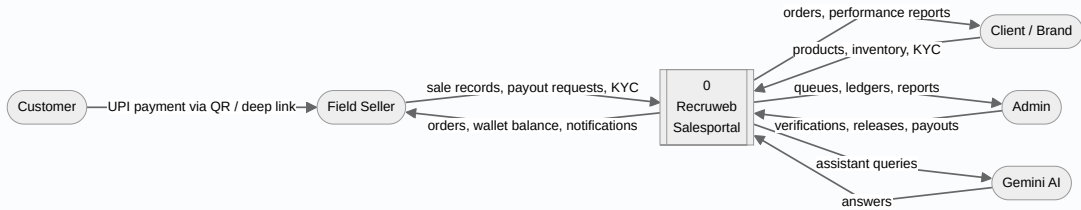


Figure 3 — DFD Level 0 (context diagram)

7.2 DFD Level 1 — Major Processes and Data Stores

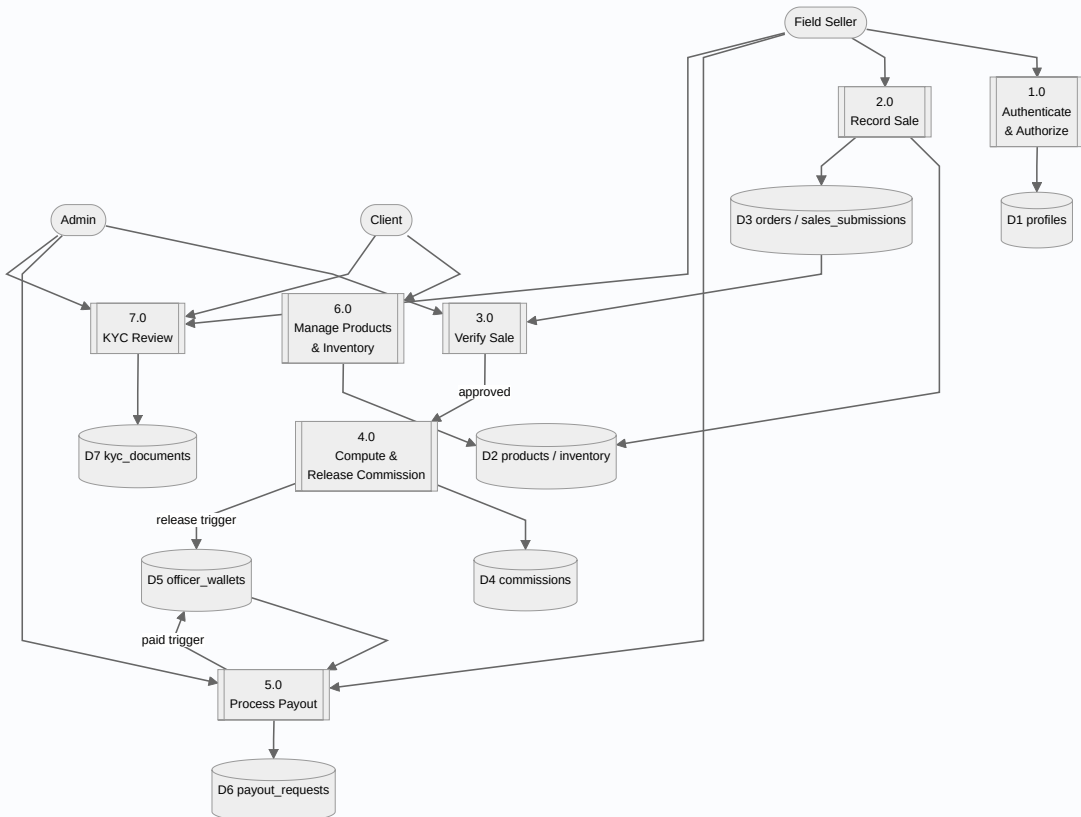


Figure 4 — DFD Level 1 (processes and data stores)

8. Database Design

The schema is applied through ordered SQL files in `backend/db/` . Key tables:

Table	Purpose	Notable columns / behavior
profiles	One row per auth user	role (admin/client/field), seller_category (field/independent), bank details, phone
products , inventory	Client catalog	price, stock; stock displayed live in the sell form
orders / sales submissions	Recorded sales	order no., customer, qty, amount, payment mode + reference, status (pending/approved/rejected)
commissions	Earnings per approved sale	officer_id → auth.users, rate, amount, status (pending/available/settled)
officer_wallets	Per-seller balance	available / pending / withdrawn; maintained by DB triggers , never written directly by app code
payout_requests	Withdrawal workflow	amount, bank/UPI snapshot, status (pending/approved/paid/rejected), admin note, transaction ref
kyc_documents	Compliance	doc type, storage path (signed URL access), review status
chat / notifications	Comms	support threads to admin inbox; in-app notifications

Design note: `commissions.officer_id` references `auth.users` rather than `profiles` ; the API therefore joins profiles in a separate batched query. This was the root cause of a production-class bug (HTTP 500 from a PostgREST embed) that was found and fixed during verification testing.

9. Key Modules — How They Work

9.1 Authentication & Role Guarding

Supabase Auth issues a JWT on login. The Next.js layout for each portal checks the session and role before rendering; the Express middleware independently re-verifies the JWT and loads the profile, rejecting cross-role API calls. The super-admin identity is pinned to a single `ADMIN_EMAIL` configured identically in both frontend and backend — a mismatch demotes admin logins to the client portal by design.

9.2 International Phone Input

A shared `PhoneInput` component (used on sell, join, and profile pages) provides a 36-country dial-code picker. Its sanitizer maps digits from 13 Unicode numeral systems (Devanagari → 981, Arabic-Indic, Bengali, Tamil, fullwidth, etc.) to ASCII before validation, so users typing on non-Latin keyboards are never silently blocked. Layout keeps the number field a minimum of 180px and wraps it to a full row when space is tight.

9.3 Live Location Detection

The Detect control calls `navigator.geolocation.getCurrentPosition` (high accuracy, 12s timeout). Coordinates are reverse-geocoded through BigDataCloud's keyless client endpoint into "City, Region". Failures degrade gracefully: raw coordinates on geocoder failure, and clear `role="alert"` messages for permission-denied, position-unavailable, or timeout.

9.4 UPI Payment Collection

When the payment mode is UPI, the `upiCollect` component builds a standard `upi://pay` URI containing the company VPA (`NEXT_PUBLIC_UPI_VPA`), payee name, exact total (recomputed from $\text{price} \times \text{quantity}$), currency INR, and an order note. It is rendered both as a QR code (customer scans with any UPI app; amount arrives locked) and as a tap-to-pay link for same-device payment. The seller then records the UPI transaction ID from the customer's confirmation as the payment reference, which the admin sees during verification.

9.5 Commission & Wallet Engine

Approving a sale creates a commission row ($\text{rate} \times \text{sale amount}$) in *pending* state. An admin "Release to Available" action flips the status, and a PostgreSQL trigger credits the seller's wallet. Payout approval similarly moves money from available to withdrawn via trigger, guaranteeing the ledger stays consistent regardless of which API path performs the update.

9.6 AI Assistant

The frontend widget posts conversation turns to `/api/assistant/chat` ; the backend enriches the prompt with role context and calls Google Gemini using a server-side key. The assistant answers questions about commissions, seller categories, and platform processes.

10. Security Measures

- JWT-verified sessions on every API request; role checked server-side, not just in the UI.
- Super-admin lockdown to a single configured email on both tiers.
- Secrets kept in gitignored `.env` files; `.env.example` templates committed instead.
- KYC documents served through short-lived signed URLs, never public buckets.
- Payment references are recorded verbatim and surfaced to the human verifier before any commission exists — money never moves on unverified input.
- Wallet arithmetic isolated in database triggers to prevent balance drift.

11. Testing & Verification Performed

The platform was verified with live browser automation against the running system:

Area	Result
Login + role redirects for all three roles	PASS
All field, admin, and client portal pages load without errors	PASS
End-to-end sale: submit → admin approve → commission created	PASS
Commission release → wallet credited → payout requested → Approve & Pay → wallet shows withdrawn	PASS
Rejection path with admin note	PASS
UPI QR renders with correct <code>upi://pay</code> parameters and live total	PASS
Geolocation success, permission-denied, and fallback paths	PASS
Unicode digit normalization (Devanagari → ASCII) in phone input	PASS
AI assistant returns live Gemini answers	PASS

Defects found and fixed during testing: (1) commissions list HTTP 500 caused by an invalid PostgREST embed — replaced with a batched profile join; (2) missing `payout_requests / officer_wallets` tables — schema applied and API cache reloaded; (3) silent blank states — explicit error UI added; (4) fake location detector — replaced with real geolocation; (5) phone field stripping non-Latin digits — normalization added.

12. Limitations & Production Roadmap

- **Hosting:** currently runs on a developer machine behind a temporary tunnel; production requires deployed frontend (Vercel) and backend (managed Node host) with a stable domain.
- **Payment verification is manual:** UPI collection is real, but confirmation trusts the typed transaction ID. A gateway (Razorpay/Cashfree) with webhook verification is recommended before real money flows at scale.
- **Notifications:** no transactional email yet (approvals, payouts, password resets).
- **Global readiness:** currency fixed to INR; UPI is India-only; no i18n; legal pages (privacy/terms) pending.
- **Operations:** add error monitoring (Sentry), uptime checks, automated DB migrations, automated tests, and CI.

13. Appendix — Environment Variables

Scope	Variable	Purpose
Backend	SUPABASE_URL , SUPABASE_SERVICE_ROLE_KEY , SUPABASE_DB_URL	Database and admin API access
Backend	ADMIN_EMAIL	Super-admin lockdown (must match frontend)
Backend	GEMINI_API_KEY	AI assistant
Frontend	NEXT_PUBLIC_SUPABASE_URL , NEXT_PUBLIC_SUPABASE_ANON_KEY	Auth client
Frontend	NEXT_PUBLIC_API_URL	Backend origin
Frontend	NEXT_PUBLIC_UPI_VPA , NEXT_PUBLIC_UPI_PAYEE_NAME	UPI payment QR payee identity